

★ CAPSTONE PROJECT — END OF TRAINING

DeskHub — Support Ticket Dashboard

A vanilla-JS, ES-module, real-API capstone for the JavaScript training program

5 days

SUGGESTED

Vanilla JS

NO FRAMEWORK

ES Modules

ARCHITECTURE

json-server

FAKE BACKEND

Why this project

Service-based work for global clients is mostly internal tools, dashboards, and CRUD admin panels — list-detail views with search, filter, sort, pagination, and form-heavy create/edit screens against REST APIs. A todo list doesn't touch most of that surface. A support ticket dashboard does — and it's the same shape as the first ticket you'll be assigned in your first client project.

By the end of this build you'll have produced a portfolio-grade, hireable artifact that exercises every concept from the 27-session program, from variables (Day 1) all the way to debounce and security (Day 27).

The fictional brief: "DeskHub" is a support ticketing app for an Indian SaaS company. Customers from across India submit support tickets; agents triage, comment, and resolve them. Admins see everything. You're building the agent/admin web app.

Tech stack

Layer	Tool	Why
Backend	json-server over db.json	Real REST endpoints, zero backend code
HTTP	fetch + async/await	What every modern app uses
Architecture	ES modules (import / export)	Day 12 + Day 24 callback
State	Plain JS objects + localStorage	No Redux — yet
UI	Hand-written HTML + CSS	Practice your fundamentals
Build	None	No bundler, no transpiler — just open the file
Dev runtime	live-server	Auto-reload on save

Why no framework? Frameworks hide the JavaScript. This project forces you to use the language directly — and once you understand it, picking up React in week one of your client project is easy.

Setup

```
# Clone the starter repo
git clone <repo-url> deskhub
cd deskhub

# Install dev dependencies
npm install

# Run API + UI together
npm run dev
```

Two services start:

Service	URL	What
json-server	http://localhost:3001	Fake REST API
live-server	http://localhost:8080	Static site, auto-reloads on save

Open `http://localhost:8080/` in your browser. (You'll need to create `public/index.html` first — see Phase 1.)

Folder structure

The starter gives you everything below except the `public/` folder. **You build `public/` from scratch — every HTML and CSS file.**

```
deskhub-starter/
├─ package.json      ← given
├─ db.json           ← given (5 users, 30 tickets, 20 comments)
├─ README.md         ← given (you'll edit it)
├─ PROJECT_GUIDE.md  ← given (this file)
├─
├─ public/           ← YOU build everything here
│   ├─ index.html     ← login page
│   ├─ dashboard.html
│   ├─ tickets.html
│   └─ ticket-detail.html
```

```

├── styles/main.css
├── src/
│   ├── main.js
│   ├── api/
│   │   ├── client.js
│   │   ├── tickets.js
│   │   └── auth.js
│   ├── modules/
│   │   ├── auth.js
│   │   ├── tickets.js
│   │   ├── ticketDetail.js
│   │   ├── form.js
│   │   └── ui.js
│   └── utils/
│       ├── debounce.js
│       ├── formatDate.js
│       └── storage.js

```

← stubs given — fill the TODOs

← fetch wrapper

← ticket CRUD

← fake login

← login UI wiring

← list page logic

← validation

← toast / modal / loader

← Day 27 callback

← Intl APIs

← localStorage wrapper

Module loading note — In your HTML, load `main.js` with `<script type="module" src="../../src/main.js"></script>`. ES modules require this attribute. Without it, you'll get "Cannot use import statement outside a module".

API cheatsheet

`json-server` gives you a full REST API. Combine query params freely.

```

# Tickets
GET    /tickets                # list all
GET    /tickets?status=open  # filter by exact match (any field)
GET    /tickets?status=open&priority=urgent # combine filters
GET    /tickets?_sort=createdAt&_order=desc # sort
GET    /tickets?_page=1&_limit=10          # paginate (also returns X-Total-Count header)
GET    /tickets?q=login                    # full-text search across all fields
GET    /tickets/12                       # single ticket
POST   /tickets                        # create (auto-assigns id)
PATCH /tickets/12                      # partial update
DELETE /tickets/12                      # delete

# Comments (filter by ticket)
GET    /comments?ticketId=12&_sort=createdAt&_order=asc
POST   /comments

# Users (for fake login)
GET    /users?email=priya@deskhub.in

```

X-Total-Count — When you paginate, json-server returns the total in the `X-Total-Count` response header. You need this for pagination UI ("Page 2 of 7"). Read it via `response.headers.get("X-Total-Count")`. **Note:** in your fetch wrapper, return both `data` AND headers, or pull it inline before calling `.json()`.

PHASE PLAN — 5 PHASES

Phase 1 — Foundations & Login

Goal: get the dev loop running, build the login flow end-to-end

- Run `npm run dev`, confirm both services are up
- Create `public/index.html` (login page) — semantic form, basic CSS
- Implement `utils/storage.js` — get / set / remove / clear with prefix
- Implement `api/client.js` — request, get, post, patch, del
- Implement `api/auth.js` — login, logout, getCurrentUser, isAuthenticated
- Implement `modules/auth.js` — wire up the form, redirect on success
- On wrong creds: show inline error, button stays enabled
- On success: store token + user, navigate to `dashboard.html`

Done when: you can log in with `priya@deskhub.in / demo123` and reach a (placeholder) dashboard. Reload — still logged in.

Phase 2 — Tickets List + API Client

Goal: build the main page — fetch and render all tickets

- Create `public/tickets.html` with a table or card grid placeholder
- Implement `api/tickets.js` — listTickets, getTicket, createTicket, updateTicket, deleteTicket, listComments, addComment
- Implement `modules/tickets.js` — initTicketsList, refresh, renderTable
- Implement `utils/formatDate.js` — formatDate, formatDateTime, formatRelative
- Show loading spinner while fetching (use ui.js stub later, or a placeholder div for now)
- Display: ID, title, customer, priority, status, assignee name, created date
- Empty state: "No tickets found" message
- Error state: friendly message + retry button

Done when: tickets.html shows all 30 tickets in a clean table with proper formatting and loading/error states.

Phase 3 — Search, Filter, Sort, Paginate

Goal: make the list usable at scale

- Add search input — debounced 300ms (use `utils/debounce.js`)

- Add status / priority / assignee dropdowns — populate dynamically from `/users`
- Add sort dropdown — by createdAt, priority, status
- Add pagination — prev / next + page numbers, 10 per page
- Read `X-Total-Count` header to compute total pages
- Reset page to 1 whenever a filter changes
- *Stretch*: reflect filters in URL query string so refresh keeps state

Done when: you can search for "login", filter to status=open, sort by priority, paginate, and the table updates correctly. No double-fetches on rapid typing (debounce works).

Phase 4 — Detail View + Create Form

Goal: full CRUD with proper form validation

- Create `public/ticket-detail.html` — read `?id=N` from URL
- Implement `modules/ticketDetail.js` — load ticket + comments in parallel
- Add status / priority / assignee dropdowns that PATCH on change
- Build the create-ticket modal (use `modules/ui.js`)
- Implement `modules/form.js` — validators + validateField + validateForm
- Validation rules:
 - title: required, 5–100 chars
 - description: required, min 10 chars
 - customerName: required
 - customerEmail: required + email format
 - priority: required, oneOf valid values
 - category: required, oneOf valid values
- Inline errors on blur, re-validate on submit, disabled submit when invalid
- On submit: POST, close modal, refresh list, show toast

Done when: you can click any ticket → see details → change status → see toast → reload page → status persisted. New tickets can be created with full validation.

Phase 5 — Comments, Polish, README

Goal: ship a polished artifact

- Add comments thread on detail page — list existing, add new
- Implement `modules/ui.js` properly — toast (auto-dismiss 3s), modal (Esc/click-outside), confirmDialog
- Delete ticket button — confirmDialog, then DELETE, then redirect
- Add the dashboard page with 4 stat cards + recent 5 tickets
- Polish CSS — consistent spacing, decent typography, responsive at 768px+
- Empty states everywhere they apply
- Update `README.md` — your own setup notes, screenshots, architecture decisions, known limitations, what you'd add next

- Final check: log in, create ticket, edit, comment, delete — full flow with no errors

Done when: you'd be happy to put this on your CV. README is a real document, not the template.

Acceptance criteria

A finished submission must have:

#	Criterion
1	<code>npm run dev</code> works on a fresh clone — no manual setup steps
2	Login + logout work; session survives page reload
3	Tickets list shows real data with at least 5 columns
4	Search input is debounced (no fetch on every keystroke)
5	At least 3 working filters (status, priority, assignee)
6	Pagination shows total pages and current page
7	Create-ticket form has inline validation; bad input is rejected
8	Detail view loads ticket + comments in one render (parallel fetch)
9	Status / priority / assignee changes persist (PATCH works)
10	Comments can be added and appear immediately
11	Loading and error states are handled visibly on every page
12	All <code>fetch</code> calls are wrapped in try/catch — no unhandled promise rejections
13	Code organised into ES modules — no single 500-line file
14	README explains setup, architecture, decisions, and known issues

Evaluation rubric

Area	Weight	What we look for
Functionality	30%	All 14 acceptance criteria pass
Code organisation	20%	ES modules used; clear separation api ↔ modules ↔ utils; no copy-paste
Async + error handling	15%	try/catch everywhere; loading states visible; user-friendly errors
Form validation	10%	Inline UX; both blur and submit validation; sensible messages
UI quality	10%	Looks professional, responsive, accessible-ish (labels, focus states)
README + git hygiene	10%	Clear setup; meaningful commits; explains decisions

Area	Weight	What we look for
Stretch goals	5%	Beyond the spec — see next section

A 15-minute walkthrough call accompanies the rubric. Each dev demos and is asked one or two "why did you do it this way" questions per area. That conversation matters as much as the score.

Stretch goals

Tackle ANY of these to push from "good submission" to "standout":

Stretch	Difficulty
URL-driven filters — <code>?status=open&page=2</code> reflects in URL, refresh-safe	Medium
Optimistic status updates — UI changes instantly, rolls back on API failure	Medium
Keyboard shortcuts — <code>/</code> focus search, <code>n</code> new ticket, <code>j</code> / <code>k</code> next/prev row	Easy
Dark mode toggle — persisted in localStorage	Easy
Bulk actions — select multiple tickets, change status in one go	Hard
CSV export of current filtered list	Medium
Activity log on detail page — timeline of all changes	Hard
Real-time poll — auto-refresh list every 30s, indicate new tickets	Medium
Accessibility audit — passes WAVE / axe with zero errors	Medium
Toast queue — show multiple toasts stacked, dismiss in order	Easy

Submission

1. Push your repo to GitHub. Public, with a real README.
2. Tag a release: `v1.0.0` with a short release-note describing what shipped vs. what's pending.
3. Send the trainer:
 - GitHub URL
 - One screenshot of the tickets list page
 - One paragraph: "What was hardest, what would I do differently next time?"
4. Be ready for the 15-minute walkthrough call.

PROGRAM RECAP — WHERE EVERYTHING SHOWS UP

Each part of this project exercises specific days from the 27-session program. If something feels rough, revisit that day's cheatsheet.

Project area	Days exercised
Variables, types, operators, template literals	Days 1–3
Conditionals, loops	Days 4–5
Functions, callbacks	Day 6
Arrays — <code>.map</code> , <code>.filter</code> , <code>.find</code> , <code>.reduce</code>	Day 7
Objects — destructuring, spread	Day 8
ES6+ — modules, default params, optional chaining	Day 9, 12
DOM manipulation	Day 10
Events, delegation	Day 11
try/catch around <code>await</code>	Day 12, 19
Closures (debounce, factory)	Day 14
<code>this</code> in event handlers / methods	Day 15
Classes (optional — for ui.js modal)	Day 17
<code>fetch</code> , async/await, error handling	Days 18–19
Event loop intuition (loading state during fetch)	Day 20
ES modules architecture	Day 24
Pure functions, immutability (filter/sort logic)	Day 25
Module / observer / factory patterns	Day 26
Debounce, security (textContent vs innerHTML), perf	Day 27

One last note. The point of this project isn't to ship something perfect. It's to make every concept from the training stick by building real software. Lean on the cheatsheets, ask each other questions, and write code you'd be proud to walk a senior engineer through.

Build something you'd be happy to show in an interview. Then go ace your first client project.